Install all required libraries

```
!pip install -q pyspark gradio pandas numpy

print("✅ All required libraries installed successfully!")
```

```
✅ All required libraries installed successfully!
```

Import libraries

```
import os
import pandas as pdw
import numpy as np
from pyspark.sql import SparkSession
from pyspark.sql.functions import col, count, isnan, when
from pyspark.ml.recommendation import ALS

print("✅ Libraries imported successfully!")
```

```
✅ Libraries imported successfully!
```

Double-click (or enter) to edit

Create Spark Session

```
spark = SparkSession.builder \
    .appName("Netflix Movie Recommendation System") \
    .master("local[*]") \
    .config("spark.driver.memory", "4g") \
    .getOrCreate()

spark.sparkContext.setLogLevel("ERROR")

print("✅ Spark Session created successfully!")
print("Spark Version:", spark.version)
```

```
✅ Spark Session created successfully!
Spark Version: 4.0.4
```

Download MovieLens dataset

```
import pandas as pd

movies_data = {
    "movieId": [
        1, 2, 3, 4, 5, 6, 7, 8, 9, 10,
        11, 12, 13, 14, 15, 16, 17, 18, 19, 20,
        21, 22, 23, 24, 25, 26, 27, 28, 29, 30
    ],

    "title": [
        "3 Idiots",
        "Dangal",
        "Jawan",
        "Pathaan",
        "KGF Chapter 1",
        "KGF Chapter 2",
        "RRR",
        "Baahubali",
        "Baahubali 2",
        "Pushpa",
        "Drishyam",
        "Drishyam 2",
        "Zindagi Na Milegi Dobara",
        "Yeh Jawaani Hai Deewani",
        "Kabir Singh",
        "Chhichhore",
        "Dilwale Dulhania Le Jayenge",
        "Kuch Kuch Hota Hai",
        "Munna Bhai M.B.B.S.",
```

```
            "Bajrangi Bhaijaan",
            "Taare Zameen Par",
            "PK",
            "Shershaah",
            "12th Fail",
            "Chak De! India",
            "Lagaan",
            "Andhadhun",
            "Stree",
            "Bhool Bhulaiyaa",
            "Gully Boy"
        ],

        "genre": [
            "Comedy Drama",
            "Sports Drama",
            "Action Thriller",
            "Action Thriller",
            "Action Drama",
            "Action Drama",
            "Action Drama",
            "Action Fantasy",
            "Action Fantasy",
            "Action Drama",
            "Crime Thriller",
            "Crime Thriller",
            "Comedy Drama",
            "Romance Drama",
            "Romance Drama",
            "Comedy Drama",
            "Romance",
            "Romance Drama",
            "Comedy Drama",
            "Drama",
            "Drama",
            "Comedy Drama",
            "Biography Drama",
            "Biography Drama",
            "Sports Drama",
            "Historical Drama",
            "Crime Thriller",
            "Horror Comedy",
            "Horror Comedy",
            "Drama"
        ]
    }

    movies_df = pd.DataFrame(movies_data)

    movies_df.to_csv("/content/netflix_movies.csv", index=False)

    print("✅ Netflix movie dataset created successfully!")
    print("✅ Total movies:", len(movies_df))
    print()
    print(movies_df.head(10))
```

```
✅ Netflix movie dataset created successfully!
✅ Total movies: 30

   movieId          title            genre
0        1       3 Idiots     Comedy Drama
1        2         Dangal     Sports Drama
2        3          Jawan   Action Thriller
3        4        Pathaan   Action Thriller
4        5   KGF Chapter 1     Action Drama
5        6   KGF Chapter 2     Action Drama
6        7            RRR     Action Drama
7        8      Baahubali   Action Fantasy
8        9    Baahubali 2   Action Fantasy
9       10         Pushpa     Action Drama
```

Create ratings dataset

```
ratings_data = {
    "userId": [
        1,1,1,1,1,
        2,2,2,2,2,
```

```
            3,3,3,3,3,
            4,4,4,4,4,
            5,5,5,5,5,
            6,6,6,6,6,
            7,7,7,7,7,
            8,8,8,8,8,
            9,9,9,9,9,
            10,10,10,10,10
        ],

        "movieId": [
            1,2,3,7,13,
            1,4,5,6,10,
            2,7,8,9,20,
            3,4,11,12,27,
            13,14,16,18,30,
            5,6,7,23,24,
            17,19,20,21,22,
            10,11,12,28,29,
            15,16,18,25,30,
            1,2,21,24,26
        ],

        "rating": [
            5,5,4,5,4,
            4,5,5,4,5,
            5,4,5,5,4,
            5,4,5,4,5,
            5,4,5,4,5,
            5,5,4,5,4,
            4,5,5,4,5,
            5,4,5,4,5,
            4,5,5,4,5,
            5,5,4,5,4
        ]
    }

    ratings_df = pd.DataFrame(ratings_data)

    ratings_df.to_csv("/content/netflix_ratings.csv", index=False)

    print("✅ Ratings dataset created successfully!")
    print("✅ Total ratings:", len(ratings_df))
    print()
    print(ratings_df.head(10))
```

```
✅ Ratings dataset created successfully!
✅ Total ratings: 50

   userId  movieId  rating
0       1        1       5
1       1        2       5
2       1        3       4
3       1        7       5
4       1       13       4
5       2        1       4
6       2        4       5
7       2        5       5
8       2        6       4
9       2       10       5
```

Load datasets into PySpark

```
movies_spark = spark.read.csv(
    "/content/netflix_movies.csv",
    header=True,
    inferSchema=True
)

ratings_spark = spark.read.csv(
    "/content/netflix_ratings.csv",
    header=True,
    inferSchema=True
)

print("✅ Movies loaded into PySpark")
print("✅ Ratings loaded into PySpark")
```

```
print("\nMovies:")
movies_spark.show(5)

print("Ratings:")
ratings_spark.show(5)
```

```
✅ Movies loaded into PySpark
✅ Ratings loaded into PySpark

Movies:
+-------+-------------+---------------+
|movieId|        title|          genre|
+-------+-------------+---------------+
|      1|     3 Idiots|   Comedy Drama|
|      2|       Dangal|   Sports Drama|
|      3|        Jawan|Action Thriller|
|      4|      Pathaan|Action Thriller|
|      5|KGF Chapter 1|   Action Drama|
+-------+-------------+---------------+
only showing top 5 rows
Ratings:
+------+-------+------+
|userId|movieId|rating|
+------+-------+------+
|     1|      1|     5|
|     1|      2|     5|
|     1|      3|     4|
|     1|      7|     5|
|     1|     13|     4|
+------+-------+------+
only showing top 5 rows
```

Check NULL values

```
print("Movies NULL values:")
movies_spark.select([
    count(when(col(c).isNull(), c)).alias(c)
    for c in movies_spark.columns
]).show()

print("Ratings NULL values:")
ratings_spark.select([
    count(when(col(c).isNull(), c)).alias(c)
    for c in ratings_spark.columns
]).show()
```

```
Movies NULL values:
+-------+-----+-----+
|movieId|title|genre|
+-------+-----+-----+
|      0|    0|    0|
+-------+-----+-----+

Ratings NULL values:
+------+-------+------+
|userId|movieId|rating|
+------+-------+------+
|     0|      0|     0|
+------+-------+------+
```

Remove duplicates

```
movies_spark = movies_spark.dropDuplicates(["movieId"])
ratings_spark = ratings_spark.dropDuplicates(["userId", "movieId"])

print("✅ Duplicate records removed!")

print("Number of movies:", movies_spark.count())
print("Number of ratings:", ratings_spark.count())
```

```
✅ Duplicate records removed!
Number of movies: 30
Number of ratings: 50
```

Display movie data

```
print("🎬 Netflix Movie Dataset")

movies_spark.show(30, truncate=False)
```

```
🎬 Netflix Movie Dataset
+-------+--------------------------+----------------+
|movieId|title                     |genre           |
+-------+--------------------------+----------------+
|1      |3 Idiots                  |Comedy Drama    |
|2      |Dangal                    |Sports Drama    |
|3      |Jawan                     |Action Thriller |
|4      |Pathaan                   |Action Thriller |
|5      |KGF Chapter 1             |Action Drama    |
|6      |KGF Chapter 2             |Action Drama    |
|7      |RRR                       |Action Drama    |
|8      |Baahubali                 |Action Fantasy  |
|9      |Baahubali 2               |Action Fantasy  |
|10     |Pushpa                    |Action Drama    |
|11     |Drishyam                  |Crime Thriller  |
|12     |Drishyam 2                |Crime Thriller  |
|13     |Zindagi Na Milegi Dobara  |Comedy Drama    |
|14     |Yeh Jawaani Hai Deewani   |Romance Drama   |
|15     |Kabir Singh               |Romance Drama   |
|16     |Chhichhore                |Comedy Drama    |
|17     |Dilwale Dulhania Le Jayenge|Romance         |
|18     |Kuch Kuch Hota Hai        |Romance Drama   |
|19     |Munna Bhai M.B.B.S.       |Comedy Drama    |
|20     |Bajrangi Bhaijaan         |Drama           |
|21     |Taare Zameen Par          |Drama           |
|22     |PK                        |Comedy Drama    |
|23     |Shershaah                 |Biography Drama |
|24     |12th Fail                 |Biography Drama |
|25     |Chak De! India            |Sports Drama    |
|26     |Lagaan                    |Historical Drama|
|27     |Andhadhun                 |Crime Thriller  |
|28     |Stree                     |Horror Comedy   |
|29     |Bhool Bhulaiyaa           |Horror Comedy   |
|30     |Gully Boy                 |Drama           |
+-------+--------------------------+----------------+
```

Display ratings

```
print("⭐ User Ratings")

ratings_spark.show(50)
```

```
⭐ User Ratings
+------+-------+------+
|userId|movieId|rating|
+------+-------+------+
|    10|      2|     5|
|     7|     21|     4|
|     5|     16|     5|
|     9|     16|     5|
|    10|     24|     5|
|     1|      7|     5|
|     1|      2|     5|
|     7|     20|     5|
|     4|     27|     5|
|     5|     30|     5|
|     4|     11|     5|
|     8|     28|     4|
|     1|      1|     5|
|     7|     22|     5|
|     6|      5|     5|
|     1|      3|     4|
|     6|     24|     4|
|     8|     12|     5|
|     7|     19|     5|
|     2|      5|     5|
|     3|     20|     4|
|     5|     13|     5|
|     8|     10|     5|
|    10|      1|     5|
|     4|      3|     5|
|     6|     23|     5|
|     6|      7|     4|
|     9|     30|     5|
|     2|      1|     4|
|     8|     11|     4|
```

```
|     2|      4|     5|
|     7|     17|     4|
|     3|      9|     5|
|     5|     18|     4|
|     2|     10|     5|
|    10|     26|     4|
|     3|      2|     5|
|     9|     18|     5|
|     9|     15|     4|
|     9|     25|     4|
|     3|      8|     5|
|     1|     13|     4|
|     8|     29|     5|
|     4|      4|     4|
|     6|      6|     5|
|     3|      7|     4|
|     5|     14|     4|
|    10|     21|     4|
|     4|     12|     4|
|     2|      6|     4|
+------+-------+------+
```

Basic Data Analysis

```
total_users = ratings_spark.select("userId").distinct().count()
total_movies = movies_spark.select("movieId").distinct().count()
total_ratings = ratings_spark.count()

print("📊 DATASET SUMMARY")
print("------------------------")
print("Total Users   :", total_users)
print("Total Movies  :", total_movies)
print("Total Ratings :", total_ratings)
```

```
📊 DATASET SUMMARY
------------------------
Total Users   : 10
Total Movies  : 30
Total Ratings : 50
```

Average Rating

```
from pyspark.sql.functions import avg

average_rating = ratings_spark.select(
    avg("rating").alias("average_rating")
)

print("⭐ Average Movie Rating:")
average_rating.show()
```

```
⭐ Average Movie Rating:
+--------------+
|average_rating|
+--------------+
|           4.6|
+--------------+
```

Prepare data for ALS

```
als_data = ratings_spark.select(
    col("userId").cast("int"),
    col("movieId").cast("int"),
    col("rating").cast("float")
)

print("✅ Data prepared for ALS recommendation model!")

als_data.show(10)
```

```
✅ Data prepared for ALS recommendation model!
+------+-------+------+
|userId|movieId|rating|
+------+-------+------+
```

```
|    10|      2|   5.0|
|     7|     21|   4.0|
|     5|     16|   5.0|
|     9|     16|   5.0|
|    10|     24|   5.0|
|     1|      7|   5.0|
|     1|      2|   5.0|
|     7|     20|   5.0|
|     4|     27|   5.0|
|     5|     30|   5.0|
+------+-------+------+
only showing top 10 rows
```

Create ALS Recommendation Model

```
als = ALS(
    userCol="userId",
    itemCol="movieId",
    ratingCol="rating",
    rank=10,
    maxIter=10,
    regParam=0.1,
    coldStartStrategy="drop",
    nonnegative=True
)

print("✅ ALS model created successfully!")
```

```
✅ ALS model created successfully!
```

Train the Model

```
model = als.fit(als_data)

print("✅ Recommendation model trained successfully!")
```

```
✅ Recommendation model trained successfully!
```

Generate Recommendations

```
user_id = 1

user_recommendations = model.recommendForUserSubset(
    ratings_spark.filter(col("userId") == user_id).select("userId"),
    5
)

print("🎬 Recommended Movies for User 1:")
user_recommendations.show(truncate=False)
```

```
🎬 Recommended Movies for User 1:
+------+--------------------------------------------------------------------------------+
|userId|recommendations                                                                 |
+------+--------------------------------------------------------------------------------+
|1     |[{2, 4.958769}, {1, 4.882021}, {5, 4.8308215}, {7, 4.756618}, {22, 4.7358103}]|
+------+--------------------------------------------------------------------------------+
```

Convert Movie IDs to Movie Names

```
recommendations = user_recommendations.collect()

if recommendations:
    recommended_movies = recommendations[0]["recommendations"]

    print("🎬 TOP 5 RECOMMENDED MOVIES")
    print("=" * 40)

    for rec in recommended_movies:
        movie_id = rec["movieId"]
        predicted_rating = rec["rating"]

        movie = movies_spark.filter(
```

```
            col("movieId") == movie_id
        ).select("title", "genre").collect()

        if movie:
            print("Movie :", movie[0]["title"])
            print("Genre :", movie[0]["genre"])
            print("Predicted Rating :", round(predicted_rating, 2))
            print("-" * 40)
```

```
🎬 TOP 5 RECOMMENDED MOVIES
========================================
Movie : Dangal
Genre : Sports Drama
Predicted Rating : 4.96
----------------------------------------
Movie : 3 Idiots
Genre : Comedy Drama
Predicted Rating : 4.88
----------------------------------------
Movie : KGF Chapter 1
Genre : Action Drama
Predicted Rating : 4.83
----------------------------------------
Movie : RRR
Genre : Action Drama
Predicted Rating : 4.76
----------------------------------------
Movie : PK
Genre : Comedy Drama
Predicted Rating : 4.74
----------------------------------------
```

Recommend movies for all users

```
all_recommendations = model.recommendForAllUsers(5)

print("✅ Recommendations generated for all users!")
all_recommendations.show(truncate=False)
```

```
✅ Recommendations generated for all users!
+------+--------------------------------------------------------------------------------------+
|userId|recommendations                                                                       |
+------+--------------------------------------------------------------------------------------+
|10    |[{5, 5.1915555}, {2, 5.00104}, {23, 4.968723}, {1, 4.9384365}, {24, 4.80298}]         |
|1     |[{2, 4.958769}, {1, 4.882021}, {5, 4.8308215}, {7, 4.756618}, {22, 4.7358103}]        |
|2     |[{10, 4.982554}, {5, 4.9087353}, {4, 4.782926}, {12, 4.6395073}, {29, 4.6256733}]     |
|3     |[{9, 4.9177017}, {8, 4.9177017}, {2, 4.8012123}, {13, 4.3203974}, {30, 4.23436}]      |
|4     |[{27, 4.963124}, {11, 4.866784}, {3, 4.826384}, {23, 4.715336}, {5, 4.6646843}]       |
|5     |[{30, 4.942793}, {16, 4.942793}, {13, 4.7914596}, {2, 4.760941}, {9, 4.7227407}]      |
|6     |[{5, 4.9776144}, {23, 4.957389}, {6, 4.7754464}, {11, 4.3232074}, {4, 4.256644}]      |
|7     |[{22, 4.96951}, {19, 4.96951}, {2, 4.9148974}, {20, 4.815498}, {1, 4.730974}]         |
|8     |[{10, 4.9624124}, {29, 4.9561677}, {12, 4.8234234}, {4, 4.3585706}, {5, 4.3045993}]   |
|9     |[{30, 4.9853354}, {16, 4.9853354}, {18, 4.793719}, {25, 3.9796536}, {15, 3.9796536}]  |
+------+--------------------------------------------------------------------------------------+
```

Create a Movie Lookup Dictionary

```
movie_lookup = {
    row["movieId"]: {
        "title": row["title"],
        "genre": row["genre"]
    }
    for row in movies_spark.collect()
}

print("✅ Movie lookup created!")
print("Total movies:", len(movie_lookup))
```

```
✅ Movie lookup created!
Total movies: 30
```

Recommendation Function

```
def recommend_movies(user_id, number_of_movies=5):
```

```
    user_df = ratings_spark.filter(
        col("userId") == int(user_id)
    ).select("userId")

    if user_df.count() == 0:
        return ["User not found. Please choose a user from 1 to 10."]

    recommendations = model.recommendForUserSubset(
        user_df,
        number_of_movies
    ).collect()

    if not recommendations:
        return ["No recommendations available for this user."]
    result = []

    for rec in recommendations[0]["recommendations"]:

        movie_id = rec["movieId"]
        predicted_rating = rec["rating"]

        if movie_id in movie_lookup:
            movie_name = movie_lookup[movie_id]["title"]
            genre = movie_lookup[movie_id]["genre"]

            result.append(
                f"🎬 {movie_name}\n"
                f"Genre: {genre}\n"
                f"Predicted Rating: {predicted_rating:.2f}"
            )

    return result


print("✅ Recommendation function created!")
```

```
✅ Recommendation function created!
```

Test the Recommendation System

```
user = 1

recommendations = recommend_movies(user, 5)

print(f"🎬 Recommended Movies for User {user}")
print("=" * 50)

for movie in recommendations:
    print(movie)
    print("-" * 50)
```

```
🎬 Recommended Movies for User 1
==================================================
🎬 Dangal
Genre: Sports Drama
Predicted Rating: 4.96
--------------------------------------------------
🎬 3 Idiots
Genre: Comedy Drama
Predicted Rating: 4.88
--------------------------------------------------
🎬 KGF Chapter 1
Genre: Action Drama
Predicted Rating: 4.83
--------------------------------------------------
🎬 RRR
Genre: Action Drama
Predicted Rating: 4.76
--------------------------------------------------
🎬 PK
Genre: Comedy Drama
Predicted Rating: 4.74
--------------------------------------------------
```

Create Netflix-Style Gradio Interface

```
import gradio as gr
```

```
def gradio_recommend(user_id):

    try:
        user_id = int(user_id)

        recommendations = recommend_movies(user_id, 5)

        return "\n\n".join(recommendations)

    except:
        return "❌ Please enter a valid User ID from 1 to 10."


with gr.Blocks(title="Netflix Movie Recommendation System") as app:

    gr.Markdown(
        """
        # 🎬 Netflix Movie Recommendation System

        ### Powered by PySpark ALS

        Select a User ID to get personalized movie recommendations.
        """
    )

    user_input = gr.Dropdown(
        choices=list(range(1, 11)),
        value=1,
        label="Select User ID"
    )

    recommend_button = gr.Button(
        "🎬 Recommend Movies"
    )

    output = gr.Textbox(
        label="Recommended Movies",
        lines=12
    )

    recommend_button.click(
        fn=gradio_recommend,
        inputs=user_input,
        outputs=output
    )

print("✅ Gradio application created!")
```

```
✅ Gradio application created!
```

Launch the App

```
app.launch(share=True)
```

```
Colab notebook detected. To show errors in colab notebook, set debug=True in launch()
* Running on public URL: https://05930fce5e9397c553.gradio.live

This share link is temporary and will last for up to 1 week (best effort). For free permanent hosting and GPU upgrades, run `gra
```



No interface is running right now

Create Vercel project

```
import os
import json
import shutil

# Create deployment folder
deployment_folder = "/content/netflix-vercel"

if os.path.exists(deployment_folder):
    shutil.rmtree(deployment_folder)

os.makedirs(deployment_folder)

print("Creating Vercel project...")
```

```
Creating Vercel project...
```

Save all recommendations as JSON

```
all_recommendations = model.recommendForAllUsers(5).collect()

recommendation_data = {}

for row in all_recommendations:

    user_id = row["userId"]

    recommendation_data[str(user_id)] = []

    for rec in row["recommendations"]:

        movie_id = rec["movieId"]
        predicted_rating = float(rec["rating"])

        if movie_id in movie_lookup:

            recommendation_data[str(user_id)].append({
                "movieId": int(movie_id),
                "title": movie_lookup[movie_id]["title"],
                "genre": movie_lookup[movie_id]["genre"],
```

```
                "rating": round(predicted_rating, 2)
            })

json_path = "/content/netflix-vercel/recommendations.json"

with open(json_path, "w", encoding="utf-8") as f:
    json.dump(recommendation_data, f, indent=2, ensure_ascii=False)

print("✅ recommendations.json created!")
print("Users:", len(recommendation_data))
```

```
✅ recommendations.json created!
Users: 10
```

Create index.html

```
html_code = r'''
<!DOCTYPE html>
<html lang="en">

<head>
    <meta charset="UTF-8">
    <meta name="viewport" content="width=device-width, initial-scale=1.0">

    <title>Netflix Movie Recommendation System</title>

    <link rel="stylesheet" href="style.css">
</head>

<body>

    <header>
        <div class="logo">NETFLIX</div>

        <div class="header-title">
            Movie Recommendation System
        </div>
    </header>


    <main>

        <section class="hero">

            <h1>🎬 Find Your Next Movie</h1>

            <p>
                Personalized movie recommendations powered by
                PySpark ALS.
            </p>

            <div class="recommend-box">

                <label for="userSelect">
                    Select User
                </label>

                <select id="userSelect">

                    <option value="1">User 1</option>
                    <option value="2">User 2</option>
                    <option value="3">User 3</option>
                    <option value="4">User 4</option>
                    <option value="5">User 5</option>
                    <option value="6">User 6</option>
                    <option value="7">User 7</option>
                    <option value="8">User 8</option>
                    <option value="9">User 9</option>
                    <option value="10">User 10</option>

                </select>

                <button onclick="getRecommendations()">
                    🎬 Recommend Movies
                </button>
```

```
            </div>

        </section>


        <section class="results">

            <h2>⭐ Recommended Movies</h2>

            <div id="movieContainer">

                <p class="loading">
                    Select a user and click Recommend Movies.
                </p>

            </div>

        </section>

    </main>


    <footer>

        <p>
            Netflix Movie Recommendation System
        </p>

        <p>
            Developed using PySpark ALS
        </p>

    </footer>


    <script src="script.js"></script>

</body>

</html>
'''

with open("/content/netflix-vercel/index.html", "w", encoding="utf-8") as f:
    f.write(html_code)

print("✅ index.html created!")
```

```
✅ index.html created!
```

CREATE CSS

```
css_code = r'''
* {
    margin: 0;
    padding: 0;
    box-sizing: border-box;
}

body {
    font-family: Arial, sans-serif;
    background: #141414;
    color: white;
    min-height: 100vh;
}


/* HEADER */

header {
    height: 70px;
    background: #000000;
    display: flex;
    align-items: center;
    padding: 0 6%;
    gap: 30px;
    border-bottom: 1px solid #222;
```

```
}

.logo {
    color: #e50914;
    font-size: 30px;
    font-weight: bold;
    letter-spacing: 2px;
}

.header-title {
    font-size: 18px;
}


/* HERO */

.hero {
    text-align: center;
    padding: 70px 20px;
    background:
        linear-gradient(
            rgba(0,0,0,0.75),
            rgba(20,20,20,1)
        );
}

.hero h1 {
    font-size: 42px;
    margin-bottom: 15px;
}

.hero p {
    color: #cccccc;
    font-size: 18px;
    margin-bottom: 35px;
}


/* BOX */

.recommend-box {
    background: #222;
    max-width: 550px;
    margin: auto;
    padding: 30px;
    border-radius: 12px;
    box-shadow: 0 10px 30px rgba(0,0,0,0.4);
}

.recommend-box label {
    display: block;
    margin-bottom: 10px;
    font-size: 16px;
}

select {
    width: 100%;
    padding: 14px;
    border-radius: 6px;
    border: none;
    margin-bottom: 15px;
    font-size: 16px;
}


button {
    width: 100%;
    padding: 14px;
    background: #e50914;
    color: white;
    border: none;
    border-radius: 6px;
    font-size: 17px;
    font-weight: bold;
    cursor: pointer;
}
```

```
button:hover {
    background: #f40612;
}


/* RESULTS */

.results {
    max-width: 1200px;
    margin: auto;
    padding: 40px 20px 80px;
}

.results h2 {
    font-size: 28px;
    margin-bottom: 25px;
}


#movieContainer {
    display: grid;
    grid-template-columns:
        repeat(auto-fit, minmax(220px, 1fr));
    gap: 20px;
}


/* MOVIE CARD */

.movie-card {
    background: #222;
    border-radius: 10px;
    padding: 22px;
    min-height: 230px;
    transition: 0.3s;
    border: 1px solid #333;
}

.movie-card:hover {
    transform: translateY(-8px);
    background: #2b2b2b;
}

.movie-number {
    color: #e50914;
    font-size: 15px;
    font-weight: bold;
    margin-bottom: 15px;
}

.movie-card h3 {
    font-size: 21px;
    margin-bottom: 15px;
}

.genre {
    color: #aaa;
    margin-bottom: 15px;
}

.rating {
    color: #ffd700;
    font-weight: bold;
    font-size: 18px;
}


/* LOADING */

.loading {
    color: #aaa;
    font-size: 17px;
}


/* FOOTER */
```

```
footer {
    text-align: center;
    padding: 30px;
    background: #000;
    color: #888;
}


/* MOBILE */

@media (max-width: 600px) {

    .hero h1 {
        font-size: 30px;
    }

    .header-title {
        display: none;
    }

    .logo {
        font-size: 25px;
    }

}
'''

with open("/content/netflix-vercel/style.css", "w", encoding="utf-8") as f:
    f.write(css_code)

print("✅ style.css created!")
```

```
✅ style.css created!
```

CREATE JAVASCRIPT

```
js_code = r'''
async function getRecommendations() {

    const userId =
        document.getElementById("userSelect").value;

    const container =
        document.getElementById("movieContainer");

    container.innerHTML =
        '<p class="loading">⏳ Loading recommendations...</p>';

    try {

        const response =
            await fetch("recommendations.json");

        const data =
            await response.json();

        const movies =
            data[userId];

        if (!movies || movies.length === 0) {

            container.innerHTML =
                "<p>No recommendations found.</p>";

            return;
        }

        container.innerHTML = "";

        movies.forEach((movie, index) => {

            const card =
                document.createElement("div");

            card.className = "movie-card";
```

```
                card.innerHTML = `

                    <div class="movie-number">
                        #${index + 1}
                    </div>

                    <h3>
                        🎬 ${movie.title}
                    </h3>

                    <div class="genre">
                        🎭 ${movie.genre}
                    </div>

                    <div class="rating">
                        ⭐ ${movie.rating}
                    </div>

                `;

                container.appendChild(card);

            });

        }

        catch (error) {

            console.error(error);

            container.innerHTML =
                "<p>❌ Unable to load recommendations.</p>";

        }

    }
    '''

    with open("/content/netflix-vercel/script.js", "w", encoding="utf-8") as f:
        f.write(js_code)

    print("✅ script.js created!")
```

```
✅ script.js created!
```

CHECK YOUR FILES

```
print("📁 Vercel Project Files")
print("=" * 40)

for file in os.listdir("/content/netflix-vercel"):
    print("✅", file)
```

```
📁 Vercel Project Files
========================================
✅ index.html
✅ recommendations.json
✅ style.css
✅ script.js
```

DOWNLOAD THE PROJECT

```
zip_path = shutil.make_archive(
    "/content/netflix-movie-recommendation",
    "zip",
    "/content/netflix-vercel"
)
```